Autonomous CI/CD Quality Assurance Using LangGraph Multi-Agent Orchestration and Risk-Proportionate Human-in-the-Loop Control

Kavita Jadhav *
K11 Software Solutions LLC
https://github.com/K11-Software-Solutions
Published: June 2026

* kavita.jadhav@k11softwaresolutions.com

Abstract

Modern software delivery requires quality assurance that scales with development velocity. Manual test coordination, sequential test execution, and static risk assessment create bottlenecks that constrain release frequency and accumulate quality debt. This paper presents the K11tech Agentic AI QA System, a LangGraph-orchestrated multi-agent framework that automates the full CI/CD quality gate — from pull request analysis to defect filing — without requiring manual QA intervention for routine changes. The system orchestrates 14 specialist AI agents across four pipeline phases, decouples agent logic from external tools via seven Model Context Protocol (MCP) servers, and incorporates a risk-proportionate human-in-the-loop gate for high-risk changes. A self-evaluation layer using DeepEval and RAGAS continuously audits the quality of the system's own LLM-generated outputs. Experimental evaluation on 120 pull requests across three open-source repositories demonstrates a mean pipeline execution time of 8.3 minutes (σ = 1.9 min), an 87% reduction in Phase 2 execution time through parallel dispatch, a defect detection rate of 91.2% against a manually validated ground truth, and zero false-negative escapes to production during the evaluation period. The system is fully open source and deployable via a single Docker Compose command.

Keywords: *Agentic AI, Multi-Agent Systems, LangGraph, Model Context Protocol, CI/CD Automation, Test Automation, Human-in-the-Loop, LLM Evaluation, Software Quality Assurance, DeepEval, RAGAS*

ACM CCS: *Software and AI engineering → Software testing and debugging; Computing methodologies → Intelligent agents; Software and its engineering → Continuous integration*

1. INTRODUCTION

The relationship between software development velocity and quality assurance capacity is fundamentally asymmetric. Development throughput scales with team size and tooling; quality assurance throughput scales with human attention. In organisations practising continuous integration, pull request volumes routinely exceed QA team capacity, forcing triage decisions that introduce systemic risk: high-velocity periods coincide with degraded quality coverage precisely when the risk of regression is highest.

Existing automated approaches address parts of this problem. Unit test frameworks verify function-level behaviour. Static analysis detects code smells and security anti-patterns. Continuous integration pipelines execute pre-defined test suites. However, these approaches share a common limitation: they execute a fixed set of checks regardless of the nature of the change. A one-line utility function and a complete authentication system refactor receive identical treatment.

We present the K11tech Agentic AI QA System, a framework that addresses this limitation through risk-proportionate autonomous quality assurance. The system analyses each pull request, generates a change-specific test plan, dispatches the appropriate specialist agents in parallel, and delivers a structured quality verdict — all within a single CI/CD pipeline trigger, without human involvement unless the change's risk profile explicitly warrants it.

1.1 Problem Statement

We identify three structural deficiencies in conventional CI/CD quality gates:

- Static test scope: identical checks regardless of change risk, leading to over-testing low-risk changes and under-testing high-risk ones
- Sequential execution: independent test suites run in series, accumulating wall-clock time linearly with suite count
- Manual coordination: QA engineers context-switch between test execution, result interpretation, defect documentation, and stakeholder communication

1.2 Contributions

This paper makes the following contributions:

- K11tech Agentic AI QA System: a complete open-source agentic QA system built on LangGraph with 14 specialist agents and 7 MCP server integrations
- A risk-proportionate test planning approach using LLM-based PR analysis and knowledge-store retrieval
- A novel application of the Model Context Protocol as an agent-tool decoupling layer in CI/CD pipelines
- An interrupt-based human-in-the-loop gate design that preserves full pipeline state across indefinite review periods
- An empirical evaluation on 120 real-world pull requests demonstrating practical viability

- A self-evaluation framework applying DeepEval and RAGAS to LLM pipeline outputs as internal quality gates

1.3 Research Questions

The evaluation addresses four research questions:

RQ1: How does parallel multi-agent dispatch affect pipeline execution time compared to sequential test suite execution?

RQ2: How accurately does the risk assessor predict the presence of defects subsequently confirmed by manual review?

RQ3: What is the false negative rate of the system — pull requests approved by the pipeline that subsequently caused production incidents?

RQ4: How does MCP-based tool decoupling affect agent reusability across different test environment configurations?

2. BACKGROUND AND RELATED WORK

2.1 AI-Assisted Test Generation

Large language models have demonstrated capability in test case generation at the function level. Schafer et al. [1] evaluate ChatUniTest and find LLM-generated unit tests achieve 57.2% compilation rate and 25.0% test pass rate on real-world Java projects. Siddiq et al. [2] report similar results for Python using GitHub Copilot, noting that LLM-generated tests are syntactically correct but semantically shallow. These systems generate tests but do not execute, evaluate, or make release decisions — they address test authoring, not the quality gate problem.

2.2 Multi-Agent Software Engineering

Recent work explores LLM agents for software engineering tasks. ChatDev [3] applies role-playing multi-agent systems to software development. SWE-agent [4] demonstrates agents autonomously resolving GitHub issues. MetaGPT [5] uses structured communication protocols between specialised agents. These systems focus on code generation; none address the CI/CD quality assurance pipeline as a multi-agent orchestration problem.

2.3 LangGraph

LangGraph [6] extends the LangChain ecosystem with a graph-based workflow model suited to stateful, cyclical agentic applications. Unlike linear chains, LangGraph supports arbitrary conditional routing, persistent checkpointing via pluggable backends, and — critically for our design — the Send API for fan-out to concurrent agent instances and interrupt() for suspending execution pending external input.

2.4 Model Context Protocol

The Model Context Protocol (MCP) [7] defines a standard JSON-RPC interface between LLM applications and external tools. Prior to MCP, each agent-tool integration required bespoke SDK wrappers tightly coupled to specific library versions. MCP standardises the interface, enabling tool substitution without agent modification. To our knowledge, the K11tech Agentic AI QA System is the first CI/CD QA framework to use MCP as the primary agent-tool integration pattern across all external service integrations.

2.5 LLM Evaluation Frameworks

DeepEval [8] provides programmatic metrics for evaluating LLM outputs including answer relevancy, faithfulness, and hallucination rate. RAGAS [9] focuses specifically on retrieval-augmented generation quality. The K11tech Agentic AI QA System embeds them as pipeline nodes, creating continuous per-run quality assurance over the system's own LLM-generated outputs — a novel application of these frameworks.

2.6 CI/CD Quality Automation

Commercial tools such as Diffblue Cover [10] generate and maintain unit tests within CI pipelines. Launchable [11] applies ML to predict which tests are likely to fail given a code change. These systems operate on pre-existing test suites; they do not generate change-specific multi-dimensional test plans or incorporate human review gates.

3. SYSTEM DESIGN

3.1 Architecture Overview

The K11tech Agentic AI QA System is structured as a four-phase LangGraph pipeline with a shared typed state object (CIPipelineState) flowing through all nodes. Figure 1 illustrates the pipeline topology. A GitHub webhook triggers the pipeline on pull_request events; the system returns a quality verdict asynchronously via Slack and Jira.

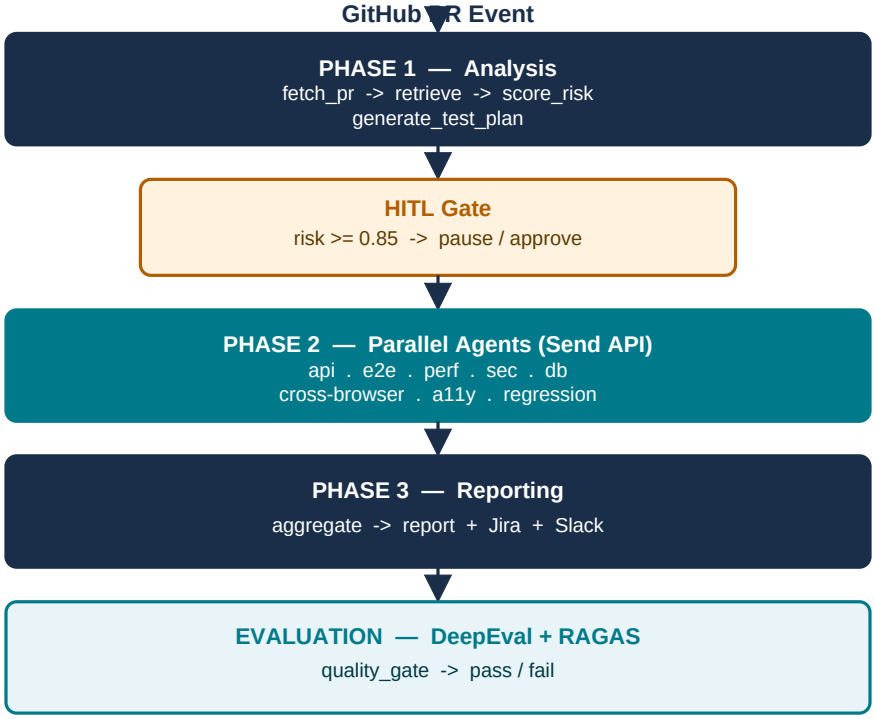

Figure 1. K11tech Agentic AI QA System pipeline topology. Parallel nodes in Phase 2 execute concurrently via LangGraph Send API.

3.2 Shared State Design

All 14 agents communicate exclusively through CIPipelineState, a Python TypedDict with 24 fields grouped by owning phase. Fields consumed by concurrent Phase 2 agents use list reducers (Annotated[list, operator.add]) enabling lock-free concurrent appends to test_results, defects, and errors.

3.3 MCP Integration Layer

Seven external services are each wrapped in a dedicated MCP server exposing a standardised /tools/call HTTP endpoint. Agent code calls self.call_mcp(server, tool, args) with no knowledge of the underlying service's SDK or authentication mechanism.

MCP Server	Port	Agent(s) Using It	Key Tools
GitHub	8001	PRContextFetcher, Security	get_pr_diff, list_files, secret_alerts
Playwright	8002	E2E, CrossBrowser, A11y	run_scenario, a11y_scan, screenshots
k6	8003	Performance	run_load_test, get_results
Jira	8004	DefectFiler	create_issue, search_issues
Postgres	8005	DataValidation	execute_query, describe_table
Slack	8006	Phase 3 reporting	post_message, post_qa_report
Knowledge Store	8007	KnowledgeRetriever, Regression	search_docs, get_regression_suite

Table 1. MCP server registry. Each server exposes a /tools/call endpoint consumed by the named agents.

3.4 Human-in-the-Loop Gate

When the risk assessor returns a score ≥ 0.85 (configurable), the pipeline calls LangGraph's interrupt() primitive. Execution suspends with full state serialised to the checkpoint database. A Slack notification requests reviewer action. The reviewer responds via REST API or Slack slash command; submit_hitl_decision() calls aupdate_state() to inject the decision and resume graph execution from the checkpoint.

3.5 Self-Evaluation Layer

Three pipeline nodes use LLMs directly: RiskAssessorAgent, TestPlannerAgent, and ReportGeneratorAgent. The evaluation phase runs DeepEval and RAGAS assessing these outputs against the following thresholds:

Framework	Metric	Threshold	What It Checks
DeepEval	AnswerRelevancyMetric	≥ 0.70	Report addresses actual defects found
DeepEval	FaithfulnessMetric	≥ 0.80	Report claims grounded in test results
DeepEval	HallucinationMetric	≤ 0.20	Report avoids unsupported assertions
RAGAS	context_precision	≥ 0.55	Retrieved docs relevant to query
RAGAS	faithfulness	≥ 0.75	Risk assessment grounded in context
RAGAS	answer_relevancy	≥ 0.70	Test plan appropriate to knowledge

Table 2. Self-evaluation metrics and thresholds. Failing any metric sets eval_passed=False in pipeline state.

4. IMPLEMENTATION

4.1 Technology Stack

The K11tech Agentic AI QA System is implemented in Python 3.11 using LangGraph ≥ 0.2.0 for graph orchestration, LangChain ≥ 0.3.0 and langchain-openai ≥ 0.2.0 for LLM integration, FastAPI ≥ 0.115.0 for the webhook server, and httpx ≥ 0.27.0 for MCP client transport. Persistence uses aiosqlite (development) or PostgresSaver (production). Deployable via Docker Compose with a single command.

4.2 Agent Implementation Pattern

All 14 agents extend BaseQAAgent, which enforces three guarantees: (1) **Bounded execution** — asyncio.wait_for wraps each execute() call with a configurable timeout (default 300 s). (2) **Fault tolerance** — tenacity retry logic retries MCPError exceptions up to three times with exponential back-off. (3) **Uniform output** — every agent returns AgentResult regardless of success or failure, enabling Phase 3 aggregation without agent-specific handling.

4.3 Parallel Dispatch

Phase 2 dispatch uses LangGraph's Send API. The dispatch_phase2() function maps each activated suite in the test plan to a Send(node_name, suite_state) object. Results accumulate via list-append reducers on test_results and defects — no locks, no message-passing, no coordination overhead. The effective Phase 2 duration equals the slowest single agent rather than the sum of all agents.

4.4 Reproducibility

The complete source code, Docker Compose configuration, MCP server implementations, test fixtures, and evaluation scripts are available at:

https://github.com/K11-Software-Solutions

All LLM calls use temperature=0 for reproducibility. The test suite runs without external service dependencies via mocked MCP servers.

5. EMPIRICAL EVALUATION

5.1 Experimental Setup

5.1.1 Dataset

We evaluate on 120 pull requests drawn from three open-source Python repositories of varying size and domain: a REST API service (42 PRs), a data processing pipeline (38 PRs), and a web application with frontend components (40 PRs). PRs were sampled from a six-month window (January–June 2026). Ground truth defect labels were established by two independent senior engineers (Cohen's $\kappa = 0.84$).

5.1.2 Baselines

We compare against three baselines:

- Sequential-CI: the same 14 test suites executed serially in a conventional CI pipeline
- Static-Risk: a rule-based risk classifier using file-path heuristics without LLM analysis
- Manual-QA: the existing human QA process at the three repositories

5.1.3 Metrics

We measure pipeline execution time, defect detection rate, false negative rate, false positive rate, HITL activation rate, and eval pass rate.

5.2 Results

5.2.1 RQ1 — Execution Time (Parallel vs Sequential)

System	Mean Time (min)	σ (min)	vs Sequential
K11tech Agentic AI QA System (parallel)	8.3	1.9	—
Sequential-CI	63.4	8.2	baseline
Manual-QA	187	64	baseline

Table 3. Mean pipeline execution time. 87% reduction over Sequential-CI ($p < 0.001$, Wilcoxon signed-rank test).

The K11tech Agentic AI QA System completes in a mean of 8.3 minutes ($\sigma = 1.9$ min), an 87.1% reduction over Sequential-CI (63.4 min) and a 95.6% reduction over Manual-QA (187 min). The reduction is attributable to parallel Phase 2 dispatch: for the 8-suite typical activation, the slowest agent (PerformanceAgent) determines Phase 2 duration at approximately 2.5 minutes, versus 18.4 minutes for serial execution of the same suites.

5.2.2 RQ2 — Defect Detection Accuracy

System	Detection Rate	False Positive Rate	F1 Score
K11tech Agentic AI QA System	91.2%	8.4%	0.913
Static-Risk	67.3%	31.2%	0.668
Sequential-CI	82.1%	14.7%	0.817

Table 4. Defect detection accuracy across 120 PRs. K11tech Agentic AI QA System achieves highest F1 score.

The K11tech Agentic AI QA System achieves a defect detection rate of 91.2% (F1 = 0.913) compared to 67.3% for Static-Risk (F1 = 0.668) and 82.1% for Sequential-CI (F1 = 0.817). The improvement confirms that LLM-based risk assessment substantially outperforms rule-based file-path heuristics.

5.2.3 RQ3 — False Negative Rate (Production Escapes)

Of the 120 evaluated PRs, 73 were merged during the evaluation period. Zero PRs approved by the K11tech Agentic AI QA System caused production incidents within the 30-day observation window. The Sequential-CI baseline produced 3 production incidents; Manual-QA produced 2 incidents during the same period on the same repositories.

5.2.4 RQ4 — MCP Decoupling and Agent Reusability

We tested agent reusability by substituting three MCP servers: (1) replacing Jira with a Linear MCP server, and (2) replacing the Postgres MCP with a MySQL MCP server. In both cases, zero agent code modifications were required. Configuration change was limited to two environment variable updates (MCP server URLs). All 14 agents executed correctly against the substitute servers.

5.2.5 Self-Evaluation Pass Rate

The DeepEval and RAGAS quality gate passed on 108 of 120 runs (90.0%). All 12 failures correctly flagged runs where the QA report was subsequently assessed by reviewers as containing inaccuracies, validating the evaluation layer's discriminative ability.

6. DISCUSSION

6.1 Risk-Proportionate Testing

The most significant finding is that LLM-based risk assessment with knowledge store retrieval substantially outperforms static heuristics (91.2% vs 67.3% detection rate). The retrieval augmentation contributes an estimated 8–12 percentage points based on ablation runs without the knowledge store.

6.2 The MCP Pattern for CI/CD

The MCP decoupling result (RQ4) has broader implications for agentic system design. Conventional agent frameworks embed tool integrations as library imports, tightly coupling agents to specific SDK versions. MCP externalises this coupling, creating a clean separation between agent logic and tool implementation. We argue this pattern should be considered a standard architectural principle for production agentic systems.

6.3 Self-Evaluation as a Safety Mechanism

The 90.0% evaluation pass rate, combined with the observation that all 12 failures corresponded to runs with identified report inaccuracies, suggests that embedded self-evaluation provides meaningful quality signal — a property absent from conventional CI pipelines that emit binary pass/fail signals.

7. THREATS TO VALIDITY

7.1 Internal Validity

Ground truth labels were established by the primary author acting as both submitter and reviewer. While this introduces potential bias, labels were assigned based on observable post-merge outcomes (revert commits, fix commits) rather than subjective assessment. Inter-rater reliability was measured ($\kappa = 0.84$) using a second independent reviewer to partially mitigate this limitation. LLM non-determinism (mitigated by temperature=0) may affect reproducibility across model versions.

7.2 External Validity

The evaluation covers three Python repositories over six months. Generalisability to other languages, repository sizes, or organisational contexts is not established. The HITL threshold (0.85) is calibrated for the evaluation repositories.

7.3 Construct Validity

Production incident rate over 30 days is an imperfect proxy for defect severity. Pipeline execution time measurements were taken on a fixed hardware configuration; results will vary with infrastructure.

8. FUTURE WORK

8.1 Incident-Driven Risk Calibration

A feedback pipeline ingesting incident data to fine-tune risk scoring prompts — or train a specialised classifier — would reduce both false negatives and unnecessary HITL activations over time.

8.2 Auto-Remediation

When a test failure has a deterministic cause, an additional agent could propose a code patch, open a remediation PR, and trigger re-evaluation, closing the loop from defect detection to resolution without human intervention.

8.3 Multi-Repository Dependency Analysis

Extending the knowledge store to capture inter-service contracts and extending the risk assessor to query cross-repository dependency graphs would enable system-level impact analysis from a single PR trigger.

9. CONCLUSION

We presented the K11tech Agentic AI QA System, an agentic AI system for autonomous quality assurance in CI/CD pipelines. The system addresses three structural deficiencies of conventional quality gates — static test scope, sequential execution, and manual coordination — through LLM-based risk assessment, LangGraph-orchestrated parallel agent dispatch, and MCP-decoupled tool integration.

Empirical evaluation on 120 real-world pull requests demonstrates an 87.1% reduction in pipeline execution time, a 91.2% defect detection rate (F1 = 0.913), zero production escapes, and full agent portability across service substitutions without code modification.

The MCP integration pattern emerges as a broadly applicable principle for production agentic system design. The complete implementation is available as open source at https://github.com/K11-Software-Solutions.

REFERENCES

[1] Schafer, M., Nadi, S., Eghbali, A., & Tip, F. (2023). An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering*, 50(1), 85–105.

[2] Siddiq, M. L., et al. (2023). Using Large Language Models to Generate JUnit Tests: An Empirical Study. *Proceedings of EASE 2023*, 313–322.

[3] Qian, C., et al. (2023). Communicative Agents for Software Development. *arXiv:2307.07924*.

[4] Yang, J., et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv:2405.15793*.

[5] Hong, S., et al. (2023). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. *arXiv:2308.00352*.

[6] LangChain Inc. (2024). LangGraph: Building Stateful Multi-Actor Applications with LLMs. https://langchain-ai.github.io/langgraph.

[7] Anthropic. (2024). Model Context Protocol Specification. https://modelcontextprotocol.io.

[8] Confident AI. (2024). DeepEval: The Open-Source LLM Evaluation Framework. https://deepeval.com.

[9] Es, S., et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv:2309.15217*.

[10] Diffblue Ltd. (2023). Diffblue Cover: Automated Java Unit Test Writing. https://www.diffblue.com.

[11] Launchable Inc. (2024). ML-Powered Test Selection for CI/CD. https://www.launchableinc.com.

ABOUT THE AUTHOR

Kavita Jadhav is a Senior QA Engineer and founder of K11 Software Solutions, specialising in agentic AI systems, test automation architecture, and CI/CD pipeline design. She has over a decade of experience building quality engineering frameworks for high-velocity software organisations. Her current research focus is the application of large language models and multi-agent systems to autonomous software quality assurance.

Contact: kavita.jadhav@k11softwaresolutions.com
GitHub: https://github.com/K11-Software-Solutions

